

Math Grapher — Bug Fixes

Seven broken/crashing equation cases and how to fix them

Bugs 1–3 came from equations you tested directly. Bugs 4–7 are predicted from the same evaluator/renderer architecture — I can't run or decompile the built app in this environment, so these are the next most likely failure points based on the pattern of the first three, not independently confirmed crashes. Worth testing each on-device.

Bug 1 — $y = x \cos(t)$ doesn't work

Cause

The equation has three symbols: y , x , and t . Your classifier treats any $y = \dots$ input as an **explicit function**, which only ever binds the variable x . The parser reads $x \cos(t)$ fine (implicit multiplication: $x * \cos(t)$), but when the evaluator tries to compute it, t has no bound value — explicit mode never gave it one. That's either:

- an unbound-variable error (if error handling exists), or
- a silent NaN / 0 result → nothing gets drawn.

Why this input doesn't really make sense as typed

An equation can't mix x and t as two independent free variables in one line — that's not a single curve, it's two unknowns tangled together.

What was probably meant

This looks like an attempt at a spiral-style curve ($t \cdot \cos(t)$ is a classic parametric shape). That needs **two** expressions, not one:

```
x = t
y = t * cos(t)
```

entered in parametric mode with a `t` range (e.g. `0` to `20`).

Fix

1. **If a plain function of x was meant** — drop the `t`: `y = x * cos(x)`.
2. **If a parametric/spiral curve was meant** — switch to parametric input and enter `x = t`, `y = t*cos(t)` as two separate expressions.
3. **Either way, harden the evaluator** so an unbound identifier never crashes silently or with an unhandled exception:

```
is AstNode.Var -> bindings[node.name]
?: throw
MathException.UndefinedVariable(node.name) // caught
above the render loop, shown as an inline error chip
```

Bug 2 — `y = b * sqrt(1 - (x^2 / a^2))` crashes the app

Cause, part 1 — `a` and `b` are undefined

Same root issue as Bug 1: `a` and `b` aren't bound to anything. If the identifier lookup is a plain map access or `.first { it.name == name }` without a null-check, it throws an **unhandled**

`NoSuchElementException` / `NullPointerException` straight out of the evaluator – and since nothing catches it above the render loop, it takes the whole app down instead of just failing to draw. This is almost certainly the actual crash.

Cause, part 2 – `sqrt` of a negative value

Even once `a / b` are bound, for $|x| > a$ the term $1 - (x^2/a^2)$ goes negative, so `sqrt(...)` returns `NaN` (Kotlin doesn't throw here). If that `NaN` gets passed straight into `Path.lineTo(px, py)`, Android's Canvas/Path can throw or corrupt the path depending on how it's called – so this equation is a two-part failure, not one.

Fix

```
// 1. Evaluator – never let an unbound identifier
throw uncaught
fun evaluate(node: AstNode, bindings: Map<String,
Double>): Double = when (node) {
    is AstNode.Var -> bindings[node.name]
    ?: throw
    MathException.UndefinedVariable(node.name)
    ...
}

// 2. Sampler – never hand NaN/Infinity to the Canvas
for (x in sampleXs) {
    val y = evaluator.evaluate(expr, mapOf("x" to x))
    if (y.isNaN() || y.isInfinite()) {
        pathStarted = false // break the path –
        start a new subpath after the gap
        continue
    }
    if (!pathStarted) { path.moveTo(x, y); pathStarted
= true }
    else path.lineTo(x, y)
}
```

This makes both failure modes fail *safely* — a caught error chip instead of a crash, and a gap in the curve instead of a corrupted draw call — rather than fixing only one and leaving the other.

What was probably meant

$y = b \sqrt{1 - (x^2/a^2)}$ is the ellipse formula, with a / b intended as **adjustable parameters**, not typos. There's currently no mechanism for a free variable with a default value. Worth giving unrecognized single-letter identifiers (other than $x / y / t / \theta$) a default value of 1 and auto-generating a draggable slider chip for each — that turns this whole class of equation from "crashes" into "works immediately and is interactive."

Bug 3 — Two-line equations aren't handled correctly

The two-line inputs in question

$x = r * \cos(t)$	$x = a * \cos(t)$
$y = r * \sin(t)$	$y = b * \sin(t)$

$y = \sqrt{r^2 - x^2}$ (x^2/a^2)	$y = b * \sqrt{1 -$ $(x^2/a^2)}$
$y = -\sqrt{r^2 - x^2}$ (x^2/a^2)	$y = -b * \sqrt{1 -$ $(x^2/a^2)}$

These look similar (both are "two lines") but they're actually **two different situations**, and the app needs to treat them differently — that's the real bug, not a single parsing error.

Case A — the $x= / y=$ pairs are genuinely ONE equation

$x = r \cos(t)$ and $y = r \sin(t)$ aren't two curves — they're one curve, defined by two coupled expressions sharing the same t . There's currently no input mode that captures two lines as a single linked equation, so each line gets submitted (and classified) separately, which doesn't work — a lone $x = r \cos(t)$ isn't graphable by itself since it has no y .

Fix: add a dedicated parametric input mode — two linked fields ($x =$ and $y =$) submitted together as one `Equation.Parametric(xExpr, yExpr, tRange)` object, producing a single chip/single curve. This is a UI addition, not something the existing single-line input can be coaxed into handling.

Case B — the $\sqrt{}$ / $-\sqrt{}$ pairs are actually TWO separate, independent equations

$y = \sqrt{r^2 - x^2}$ and $y = -\sqrt{r^2 - x^2}$ are each, on their own, a completely valid explicit function (top half / bottom half of a circle). They don't need linking to work — submitted as two ordinary single-line equations, each already renders correctly as its own chip and curve (assuming Bugs 1 & 2's fixes are in place, since these still hit undefined-variable errors for r / a / b until those are bound or slider-defaulted).

Fix (optional, cosmetic): let the app detect the $f(x)$ / $-f(x)$ pattern and offer to group the two chips — shared color, one visibility toggle for both — purely for UX. Nothing functional is broken here once Bugs 1 & 2 are fixed.

Summary

Input	What it is	What's actually needed
$x = \dots,$ $y = \dots$ (shared t)	One equation, two coupled expressions	New parametric dual-input mode (real fix, required)

Input	What it is	What's actually needed
<code>y=f(x) ,</code> <code>y=-f(x)</code>	Two independent equations	Already works once Bugs 1 & 2 are fixed; grouping is polish only

Bug 4 — Odd roots / fractional exponents of negative numbers come out wrong

Example

`y = x^(1/3)` (cube root) — mathematically defined for every real `x`, including negatives: `(-8)^(1/3) = -2`.

Cause

Kotlin's `Math.pow(base, exponent)` returns `NaN` whenever `base` is negative and `exponent` is non-integer — it doesn't know it's looking at a cube root specifically, it just sees a fractional power of a negative number and bails. So the entire left half of the curve (`x < 0`) silently vanishes, even though it's real and well-defined. This is different from Bugs 1–2: there, `NaN` was the *correct* answer for an undefined region; here `NaN` is *wrong* — the value exists, `pow()` just can't compute it.

Fix

Special-case odd-integer roots before falling through to `Math.pow`:

```
fun power(base: Double, exp: Double): Double {
    if (base < 0 && exp.isOddIntegerReciprocal()) {
        // e.g. exponent == 1/3, 1/5, ...
        return -Math.pow(-base, exp)
    }
}
```

```
        return Math.pow(base, exp)
    }
```

Bug 5 — Near-asymptote values create a fake vertical spike line

Example

```
y = 1/(x-2) , y = tan(x)
```

Cause

The NaN/Infinity gap-breaking fix (Bug 2) only catches the exact undefined point. Samples just next to it (e.g. $x = 1.999$, $x = 2.001$) are still finite — just enormous ($y \approx \pm 100000$). `lineTo()` happily connects the last low point to the first huge point, drawing a near-vertical line straight across the visible graph that looks like part of the curve but is actually a rendering artifact, not the asymptote itself.

Fix

Treat "way outside the visible y-range" the same as a gap, not just literal NaN / Infinity:

```
val ySpan = viewport.yMax - viewport.yMin
val outOfRangeThreshold = ySpan * 50 // tune as
needed
if (y.isNaN() || y.isInfinite() || kotlin.math.abs(y)
    > outOfRangeThreshold) {
    pathStarted = false
    continue
}
```

Bug 6 — Implicit equations like $x^2 + y^2 = 25$ likely don't graph at all yet

Cause

Per the architecture, implicit-equation support (marching squares) is a distinct renderer that has to be explicitly built — the classifier has to recognize "can't be isolated to $y = \dots$ " and route to it. If that renderer hasn't been implemented yet, a circle/ellipse/hyperbola typed in standard form either throws a parse error (can't isolate y) or, worse, gets mis-classified as explicit and silently fails. This isn't a crash so much as a missing feature masquerading as a bug — worth confirming whether the implicit renderer from the equation-types plan actually made it into this build.

Bug 7 — Trig functions may silently assume the wrong angle unit

Example

$y = \sin(x)$ graphed expecting x in degrees (common at the point in a course where degrees are taught before radians) will look completely wrong if the evaluator's $\sin / \cos / \tan$ always assume radians — not a crash, but a "why does this look nothing like the textbook graph" bug that's easy to misdiagnose as a parser problem.

Fix

Add an explicit degrees/radians toggle (mentioned as an overflow-menu setting in the original UI plan) and convert at evaluation time:

`sin(x)` in degree mode evaluates as `sin(x * PI / 180)` internally, not as a change to the parser.